

Cosine Similarity & Dot Products, Worked Out by Hand

Semantic search ranks documents by the angle between embedding vectors. We compute dot products, norms, cosines, and the recovered angles for one query against three documents.

What this document does. A vector-backed RAG server embeds the query and each document as vectors, then ranks documents by *cosine similarity*. This document fixes one query and three short documents as explicit vectors and computes every dot product, norm, cosine, and angle by hand — so the ranking falls out of arithmetic, not magic.

Concepts covered: embeddings as vectors · the dot product as alignment · the L_2 norm · cosine similarity · recovering the angle · why normalisation matters · dot product = cosine when unit-length.

Internal learning material. Numbers reproducible from the appendix code. v1.0

1 The setup: one query, three documents

Real embeddings have hundreds or thousands of dimensions; we use $d = 3$ so every number is visible. The query and three candidate chunks embed to:

$$q = (2, 1, 0) \text{ “database error”, } d_1 = (2, 2, 0) \text{ “sql connection failed”,}$$

$$d_2 = (0, 1, 2) \text{ “payment processed”, } d_3 = (3, 1, 0) \text{ “db timeout”}.$$

Each axis is a latent semantic feature. The goal: rank d_1, d_2, d_3 by how “close” they are to q in meaning.

Intuition

An embedding places meaning in space: texts about the same thing land in the same direction. “Close in meaning” becomes “small angle between vectors.” Cosine similarity reads that angle directly — it ignores how *long* the vectors are and asks only whether they *point* the same way.

2 Step 1 — The dot product (alignment)

The dot product of $a = (a_1, \dots, a_d)$ and $b = (b_1, \dots, b_d)$ is

$$a \cdot b = \sum_{i=1}^d a_i b_i.$$

It is large when the two vectors share large components on the same axes. Compute $q \cdot d_k$:

$$q \cdot d_1 = 2 \cdot 2 + 1 \cdot 2 + 0 \cdot 0 = 6, \quad q \cdot d_2 = 2 \cdot 0 + 1 \cdot 1 + 0 \cdot 2 = 1, \quad q \cdot d_3 = 2 \cdot 3 + 1 \cdot 1 + 0 \cdot 0 = 7.$$

Mini-example — this operation in isolation

The dot product is the entire notion of “relevance” before normalising. With $a = (1, 1)$: $a \cdot (1, 1) = 2$ (aligned, high), $a \cdot (1, -1) = 0$ (perpendicular, neutral), $a \cdot (-1, -1) = -2$ (opposed, low). Same direction $\Rightarrow$ large positive; orthogonal $\Rightarrow$ zero; opposite $\Rightarrow$ negative.

Watch out

Raw dot product secretly rewards *length*. d_3 scores 7 partly because it is a longer vector, not only because it aligns with q . Ranking on the bare dot product lets “big” documents beat genuinely relevant small ones. Cosine fixes this by dividing the length out.

3 Step 2 — The norm (length)

The L_2 norm (Euclidean length) is

$$\|a\| = \sqrt{a \cdot a} = \sqrt{\sum_i a_i^2}.$$

$$\|q\| = \sqrt{4 + 1 + 0} = \sqrt{5} = 2.236, \quad \|d_1\| = \sqrt{8} = 2.828, \quad \|d_2\| = \sqrt{5} = 2.236, \quad \|d_3\| = \sqrt{10} = 3.162.$$

4 Step 3 — Cosine similarity

Cosine similarity divides the dot product by both lengths — it is the cosine of the angle θ between the vectors:

$$\cos \theta = \frac{a \cdot b}{\|a\| \|b\|} \in [-1, 1].$$

$$\cos(q, d_1) = \frac{6}{2.236 \cdot 2.828} = \frac{6}{6.324} = 0.949,$$

$$\cos(q, d_2) = \frac{1}{2.236 \cdot 2.236} = \frac{1}{5.000} = 0.200, \quad \cos(q, d_3) = \frac{7}{2.236 \cdot 3.162} = \frac{7}{7.071} = 0.990.$$

doc	$q \cdot d$	$\ d\ $	$\cos \theta$	rank
d_3 “db timeout”	7	3.162	0.990	1
d_1 “sql connection failed”	6	2.828	0.949	2
d_2 “payment processed”	1	2.236	0.200	3

The server returns d_3, d_1 and drops d_2 . “db timeout” is the closest match to “database error” — and notice cosine put d_3 *above* d_1 even though their raw dot products (7 vs 6) were close, because d_3 aligns more tightly in direction.

5 Step 4 — Recovering the angle

Cosine is an abstract score; the angle makes it concrete via $\theta = \arccos(\cos \theta)$:

$$\theta_3 = \arccos(0.990) = 8.1^\circ, \quad \theta_1 = \arccos(0.949) = 18.4^\circ, \quad \theta_2 = \arccos(0.200) = 78.5^\circ.$$

d_3 sits 8° off the query; d_2 is nearly perpendicular (78°) — almost unrelated.

Intuition

Cosine = 1 means 0° (identical direction, maximal similarity); cosine = 0 means 90° (orthogonal, unrelated); cosine = -1 means 180° (opposite). Most text embeddings live in a cone where cosines are positive, so in practice the useful band is roughly 0.7–1.0, and a retrieval threshold (say “keep $\cos \geq 0.75$ ”) is just a maximum-angle cutoff.

6 Step 5 — Why normalisation lets you skip a step

If vectors are pre-normalised to unit length ($\|a\| = \|b\| = 1$), then

$$\cos \theta = \frac{a \cdot b}{1 \cdot 1} = a \cdot b.$$

So on a normalised index, the dot product *is* the cosine — which is why production vector stores normalise once at index time and then rank by plain dot product (one multiply-add per dimension, no per-query square roots).

Mini-example — this operation in isolation

Normalise $q = (2, 1, 0)$: divide by $\|q\| = 2.236$ to get $\hat{q} = (0.894, 0.447, 0)$, which has length $\sqrt{0.894^2 + 0.447^2} = \sqrt{1.000} = 1$. Now $\hat{q} \cdot \hat{d}_3$ returns 0.990 directly — the same cosine, no division at query time.

7 The argument, at a glance

#	Idea	Formula	Value (best doc d_3)
1	Dot product	$a \cdot b = \sum a_i b_i$	7
2	Norm	$\ a\ = \sqrt{a \cdot a}$	$\ d_3\ = 3.162$
3	Cosine similarity	$\frac{a \cdot b}{\ a\ \ b\ }$	0.990
4	Angle	$\arccos(\cos \theta)$	8.1°
5	Unit-length shortcut	$\cos \theta = a \cdot b$	if $\ a\ = \ b\ = 1$

8 Equation cheat-sheet

Dot product	$a \cdot b = \sum_i a_i b_i$
L_2 norm	$\ a\ = \sqrt{\sum_i a_i^2}$
Cosine similarity	$\cos \theta = (a \cdot b) / (\ a\ \ b\)$
Angle	$\theta = \arccos(\cos \theta)$
Normalised case	$\cos \theta = \hat{a} \cdot \hat{b}$, $\hat{a} = a / \ a\ $
Retrieval threshold	keep doc iff $\cos \theta \geq \tau$

A Reference implementation

```
import math

def dot(a, b): return sum(x*y for x, y in zip(a, b))
def norm(a): return math.sqrt(dot(a, a))
def cosine(a, b): return dot(a, b) / (norm(a) * norm(b))

q = (2, 1, 0)
docs = {"d1": (2,2,0), "d2": (0,1,2), "d3": (3,1,0)}
for name, d in docs.items():
    c = cosine(q, d)
    print(name, round(c, 3), round(math.degrees(math.acos(c)), 1))
# d1 0.949 18.4
# d2 0.2 78.5
# d3 0.99 8.1
```

— end of document —